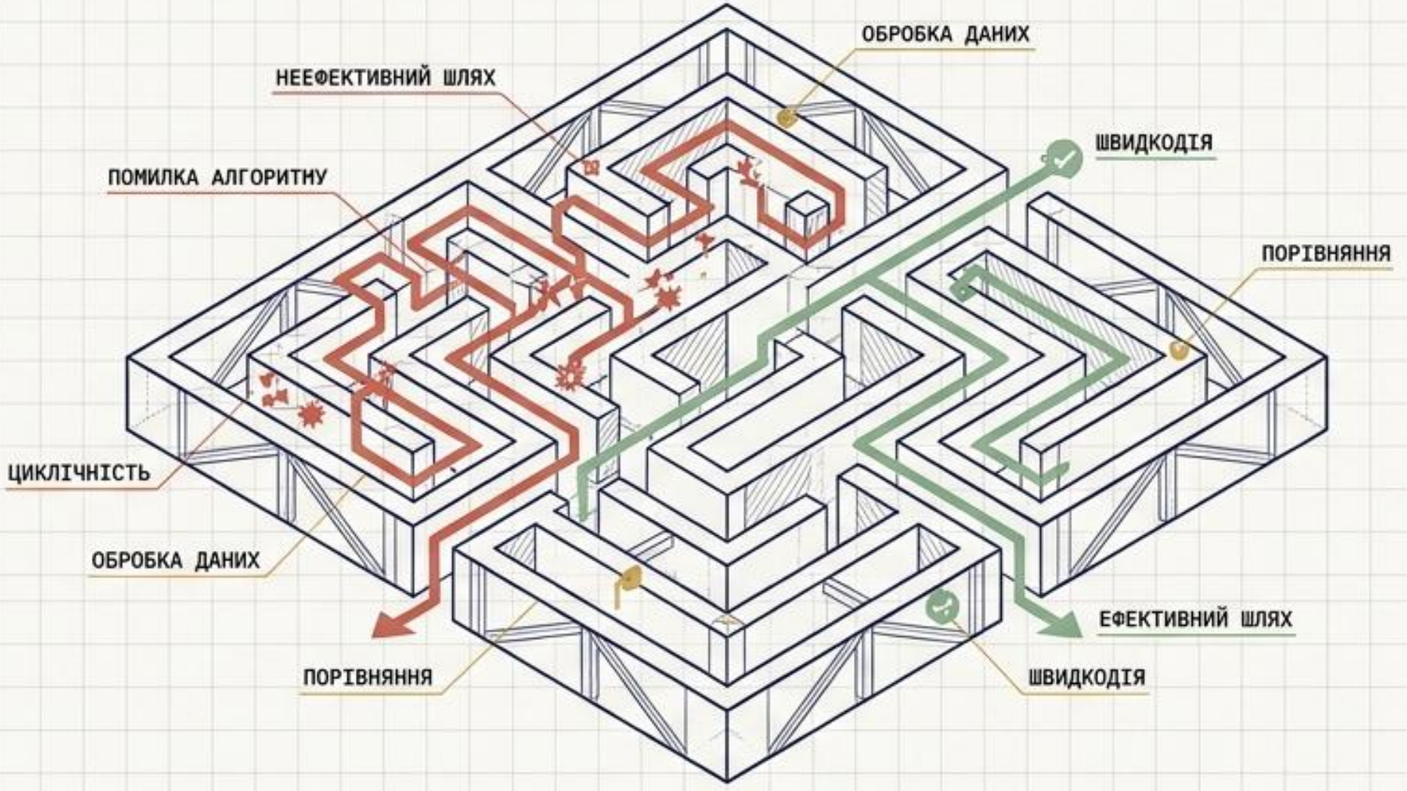
Архітектура мислення

Вступний урок до алгоритмів та структур даних.



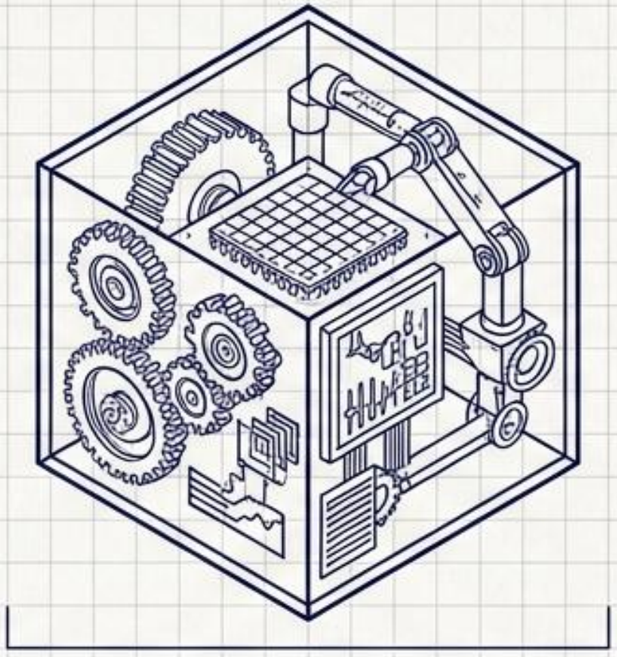
Як програми масштабуються від 10 до 10 мільйонів користувачів без збоїв.

Gemini Notebook

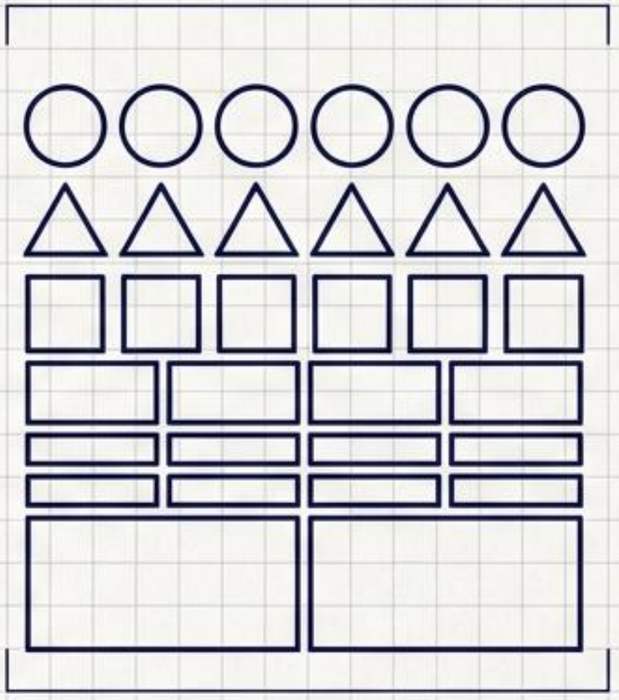
Алгоритм — це відповідь на питання «Як саме ми досягаємо результату?»



Хаос
(Вхідні дані)



Алгоритм
(Чітка послідовність кроків)

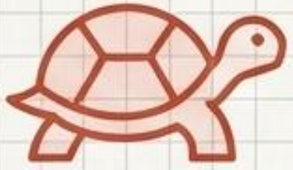


Порядок
(Результат)

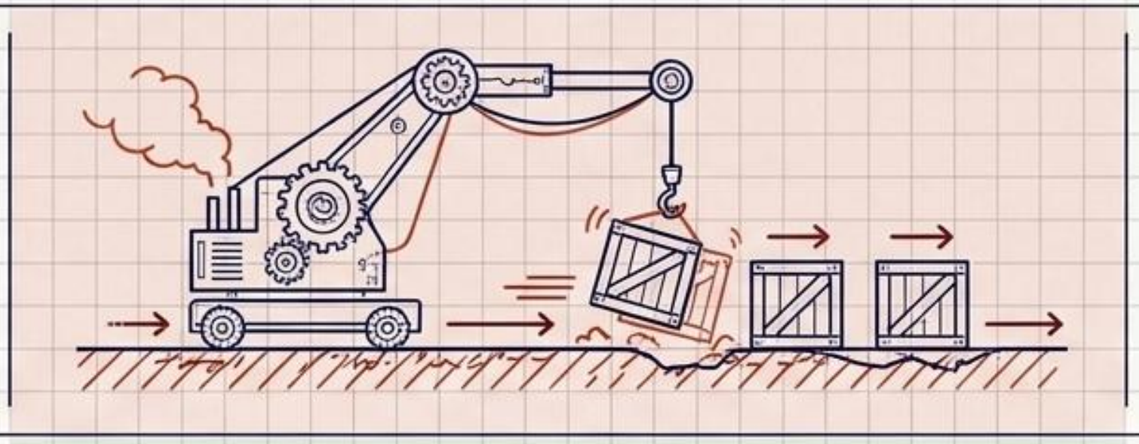
Один і той самий результат можна отримати різними шляхами. Різниця завжди полягає у витраченому часі та ресурсах комп'ютера.

Gemini Notebook

Ціна неефективності: Чому правильний вибір критично важливий



Поганий алгоритм – $O(n^2)$



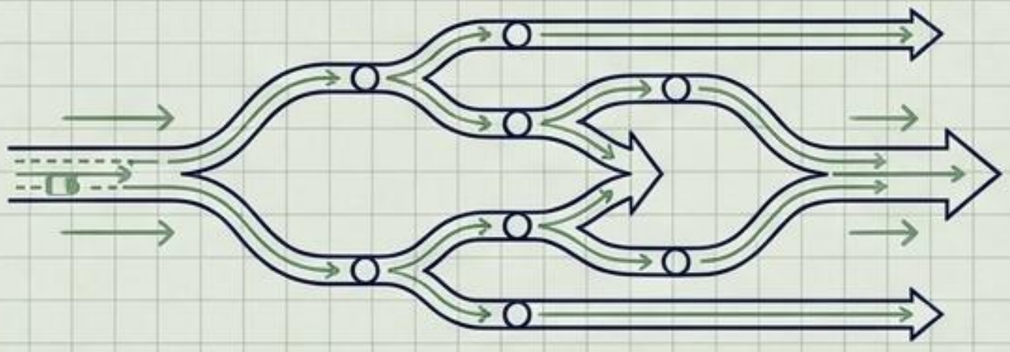
10^{12}
(1 000 000 000 000 кроків)

Кілька днів

Перевантажено ⚠



Хороший алгоритм – $O(n \log n)$



19,931,568 кроків

Частки секунди

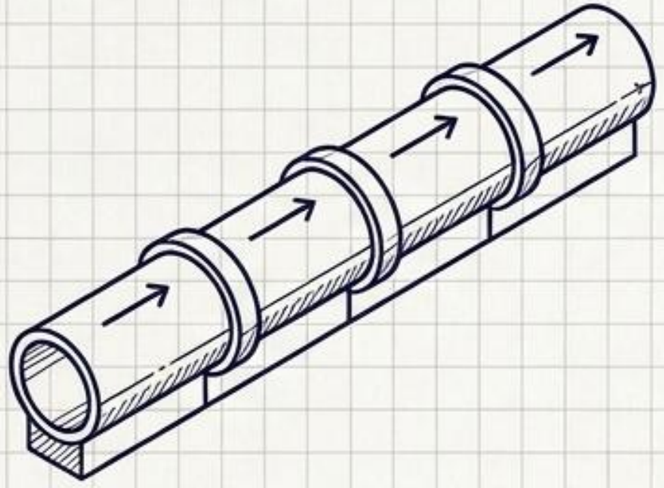
Ресурси вільні ✓

Алгоритми визначають межу між продуктом, який працює, і продуктом, який "падає" під навантаженням.

Gemini Notebook

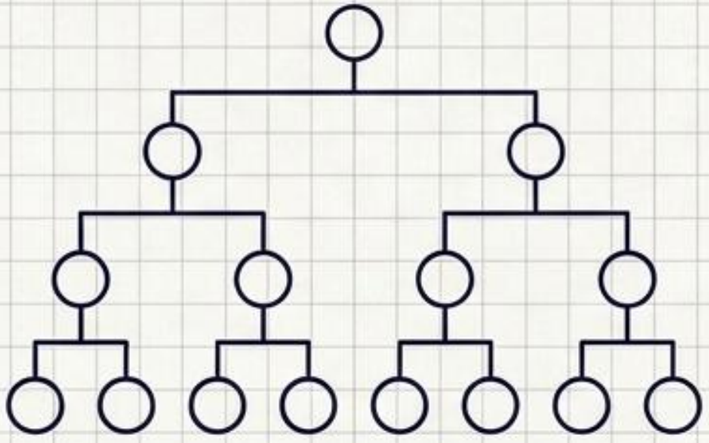
Ландшафт: Структури даних — це дороги, алгоритми — це транспорт

Лінійні
(Lists, Stacks, Queues)



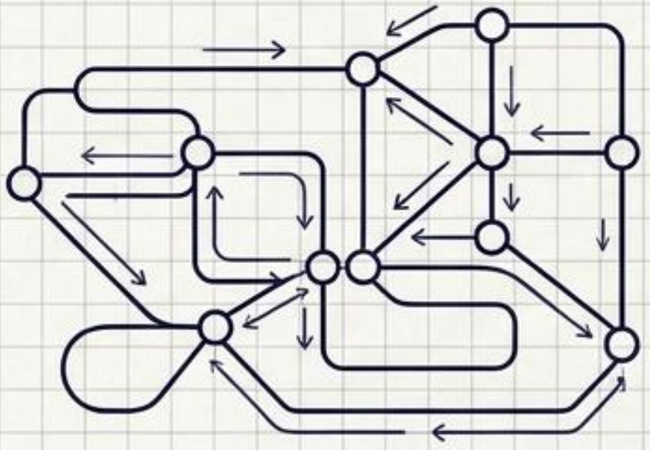
Застосування: Черги задач, історія дій.

Ієрархічні
(Trees)



Застосування: Файлові системи, індекси баз даних.

Мережеві
(Graphs)



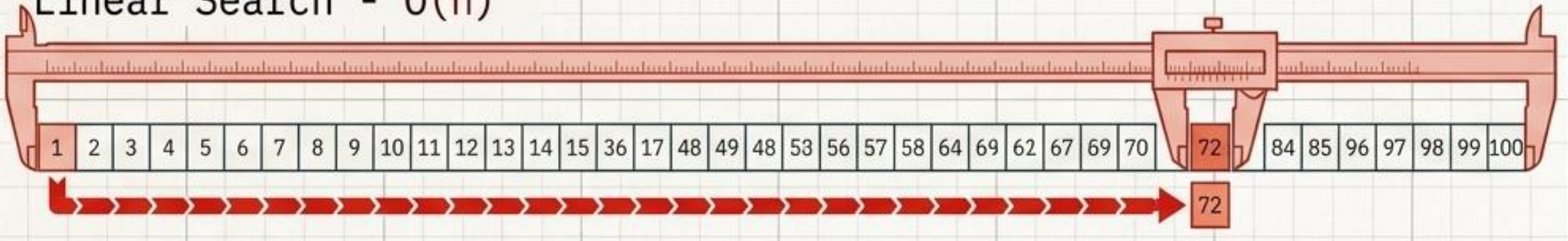
Застосування: Соціальні мережі, маршрутизація, рекомендації.

Щоб їхати швидко, недостатньо мати спортивне авто (швидкий алгоритм). Потрібно, щоб воно їхало по ідеальному шосе (правильній структурі даних).

Gemini Notebook

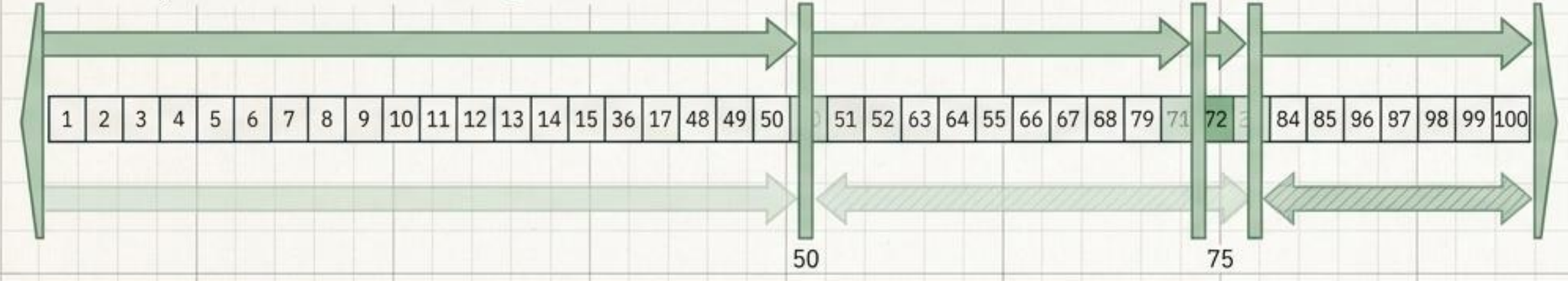
Алгоритми пошуку: Навігація в даних

Linear Search - O(n)



- Кроки для знаходження: 72 кроки.
- Особливість: Працює з невідсортованим хаосом.

Binary Search - O(log n)

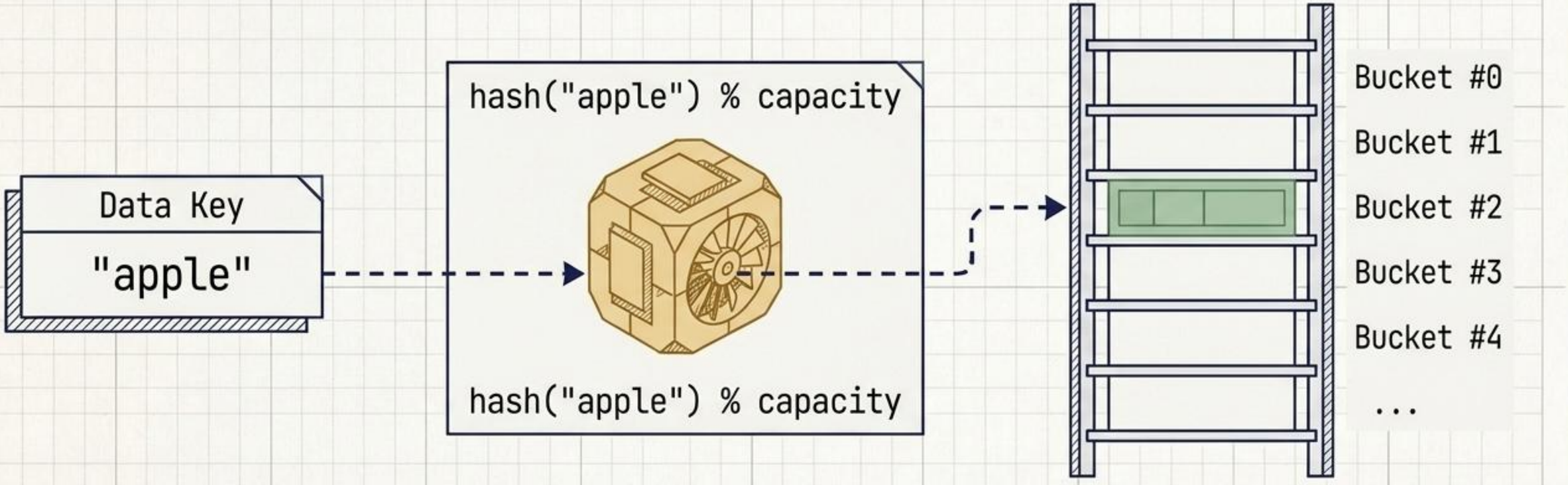


- Кроки для знаходження: ~6 кроків.
- Вимога: Дані ПОВИННІ бути відсортовані.

При 1 мільярді записів (10^9):
Лінійний пошук зробить **1 мільярд кроків**.
Бінарний знайде результат максимум за **30 кроків**.

Gemini Notebook

Хешування: Телепортація замість пошуку (O(1))



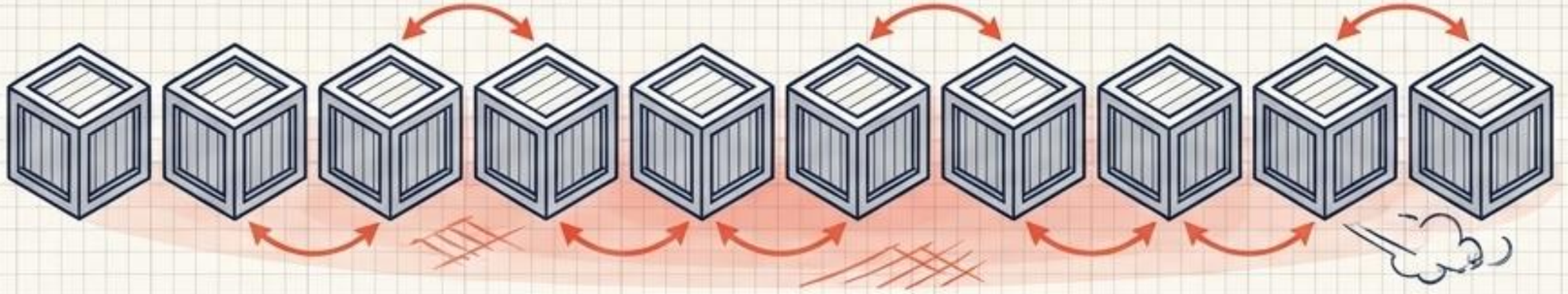
Як це працює:
Ми не шукаємо елемент. Хеш-функція математично обчислює точну адресу в пам'яті.

Ціна швидкості:
O(1) вимагає більше оперативної пам'яті (щонайменше 1/3 масиву завжди порожня для уникнення колізій).

Gemini Notebook

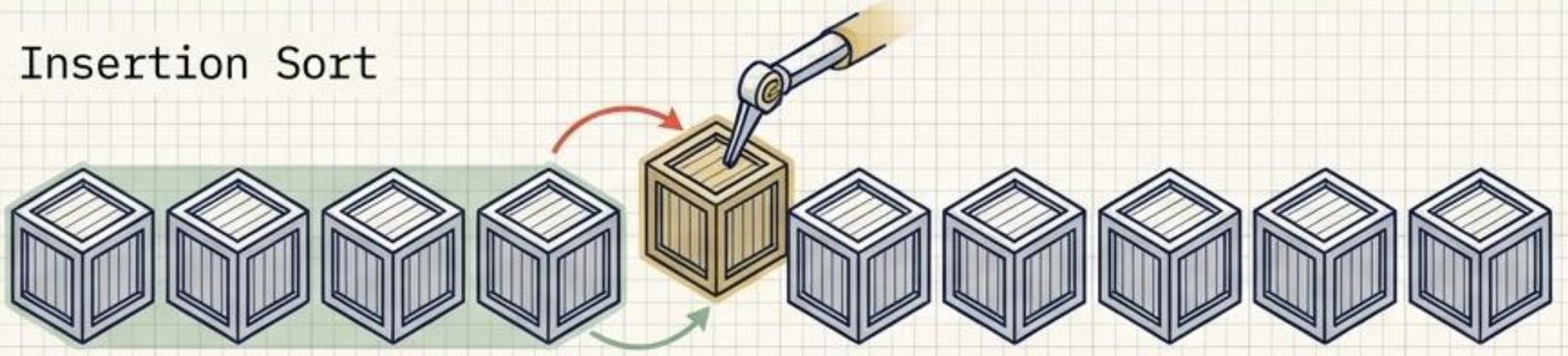
Сортування: Від хаосу до порядку (Наївні підходи)

1. Bubble Sort



Діагноз: $O(n^2)$. Постійно "тягає" важкі елементи в пам'яті. Зайва робота.

2. Insertion Sort



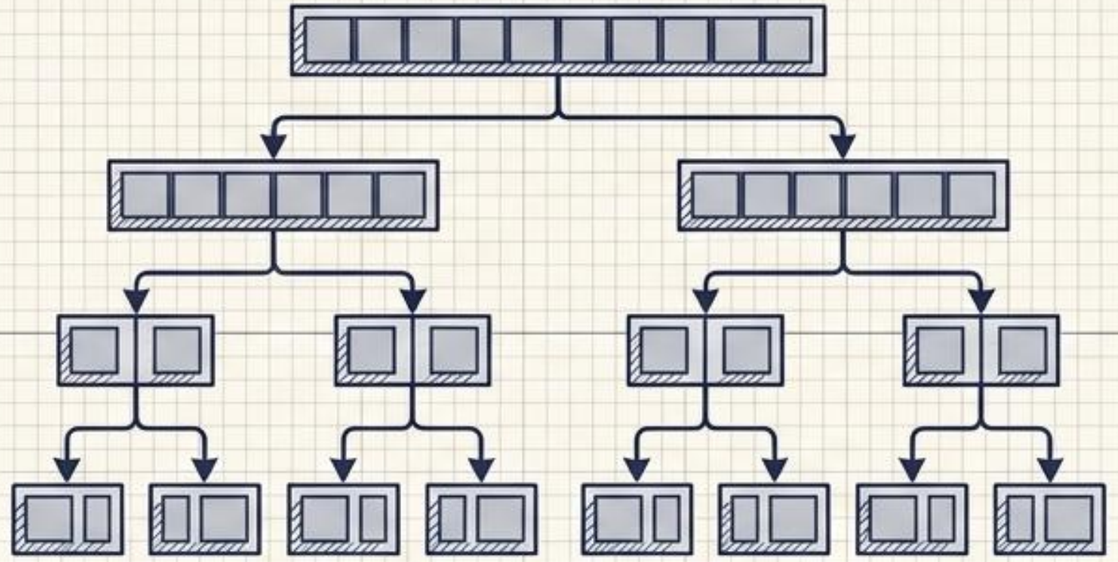
Діагноз: Чудово працює для маленьких масивів або майже відсортованих даних, але "ламається" на великому хаосі.

Правило індустрії: Сортування з часом $O(n^2)$ (Bubble, Selection) практично ніколи не використовуються в production для великих даних.

Gemini Notebook

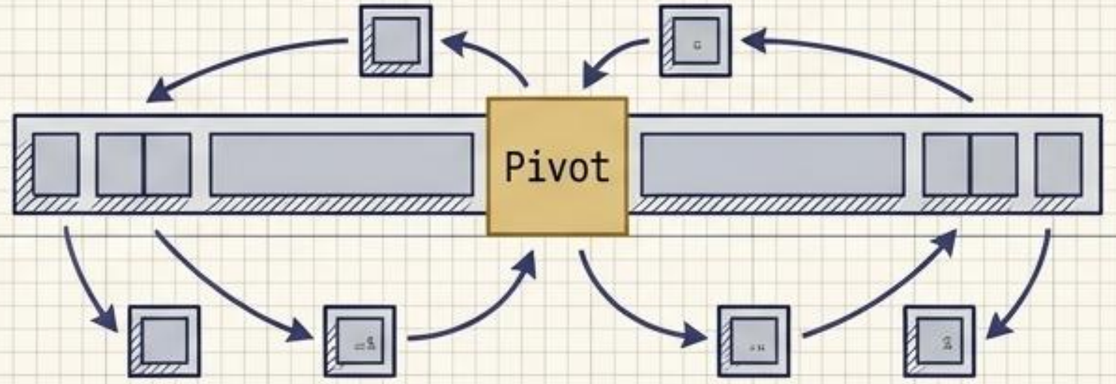
Розділяй і володарюй: Промисловий стандарт (O(n log n))

Merge Sort



Особливість: Гарантована стабільність. Добре для гігантських баз даних, але вимагає багато додаткової пам'яті.

Quick Sort



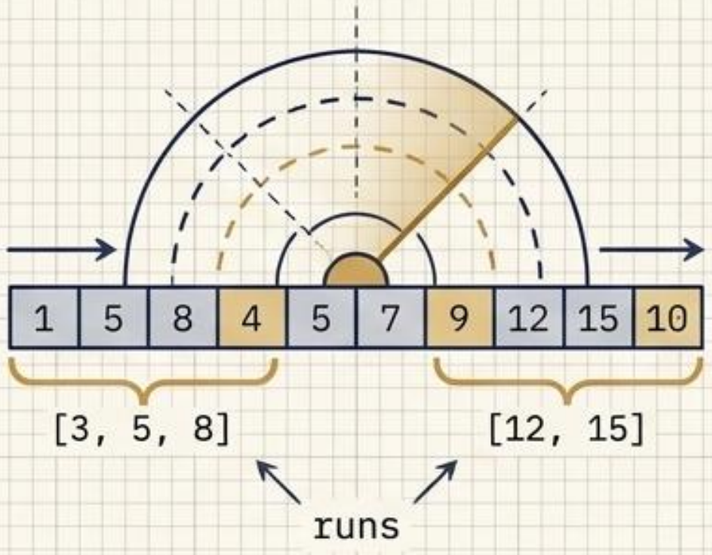
Особливість: Найшвидший на практиці завдяки ідеальному використанню кешу процесора (читає дані послідовно).

Це не просто прискорення коду.
Це повна зміна архітектури процесу.

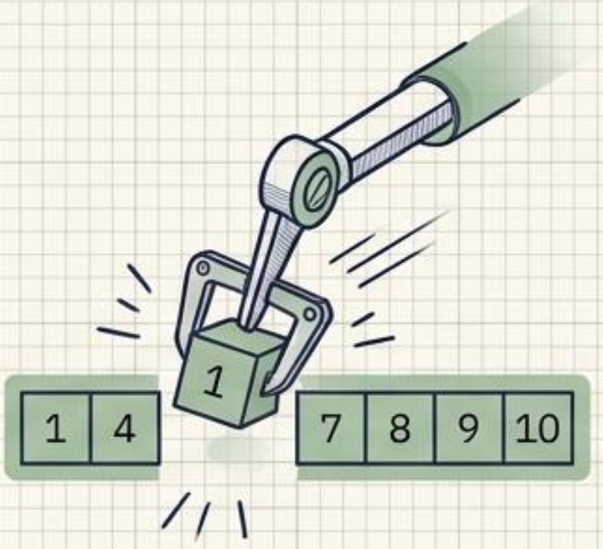
Gemini Notebook

Як сортує Python? Знайомтесь: Timsort

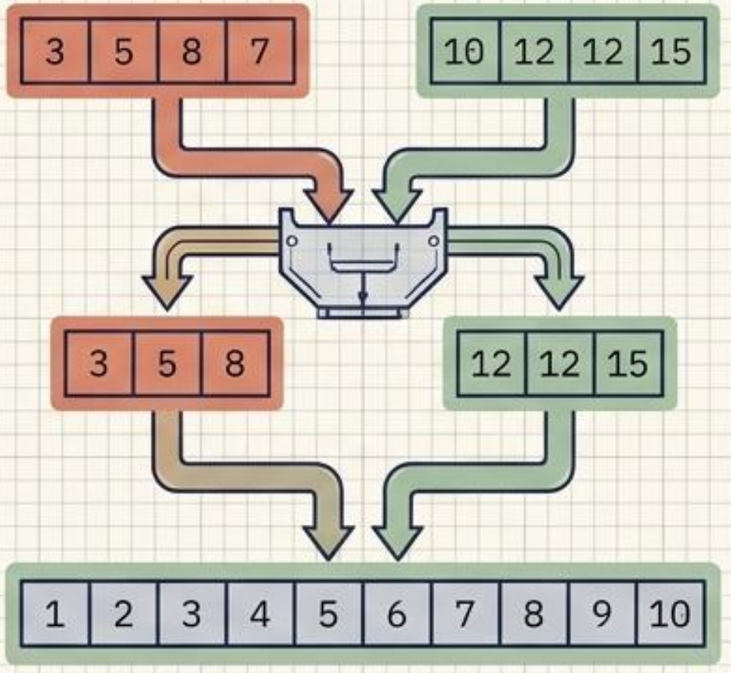
1. Сканування (Scan):



2. Оптимізація (Insertion Sort):



3. Злиття (Merge):

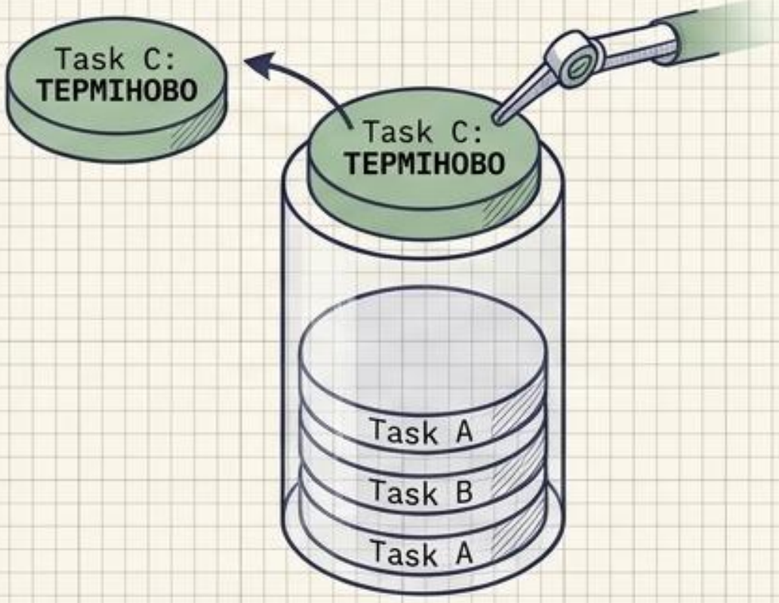


Timsort – це гібридний алгоритм. Він не намагається бути 'найрозумнішим'. Він адаптується до реальних даних користувача, шукаючи закономірності (вже відсортовані фрагменти), щоб уникнути зайвої роботи.

Gemini Notebook

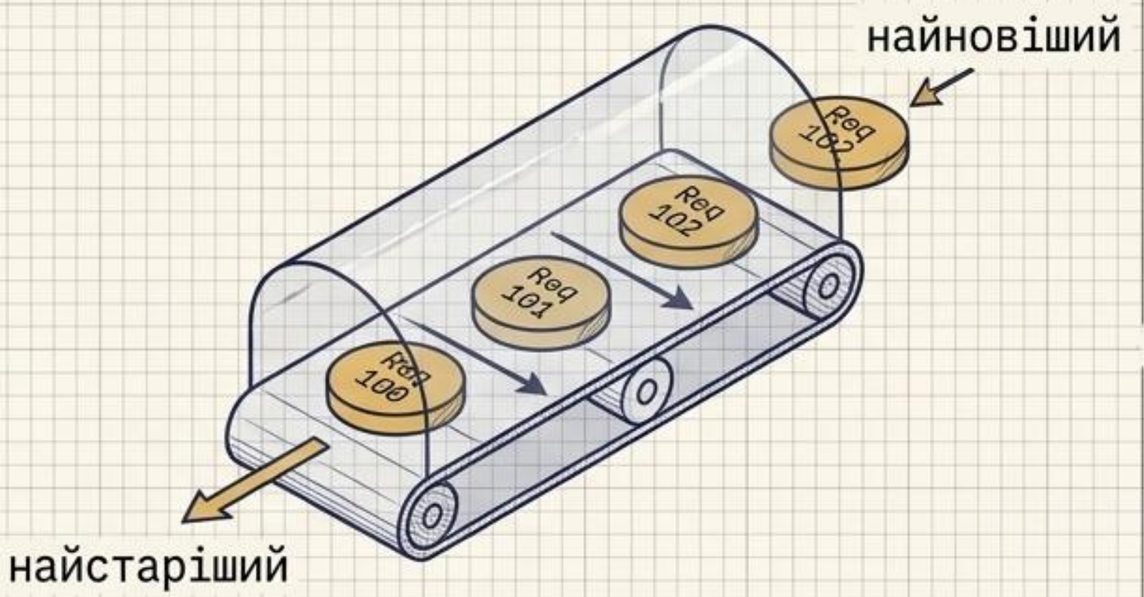
Управління потоком даних: LIFO проти FIFO

Stack (LIFO - Last-In, First-Out)



Де працює: Скасування дій (Undo), виклики функцій (Рекурсія), обхід дерев у глибину (DFS).

Queue (FIFO - First-In, First-Out)



Де працює: Черги запитів до сервера, черги на друк, обхід у ширину (BFS).

Стек та черга – це фундаментальні абстрактні типи даних, що визначають порядок обробки. Вибір залежить від того, чи потрібно пріоритизувати останнє додане (LIFO) (IFO) або підтримувати хронологію (FIFO).

Gemini Notebook

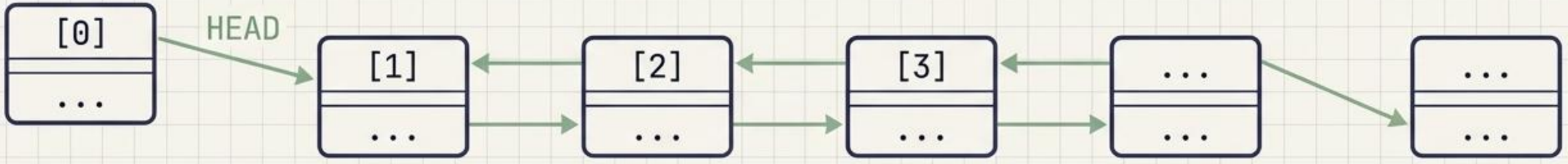
Архітектурна катастрофа: чому list.pop(0) вбиває сервер

Погана практика (list)



Складність: O(n) — процесор виконує мільйон зайвих рухів.

Правильне рішення (collections.deque)



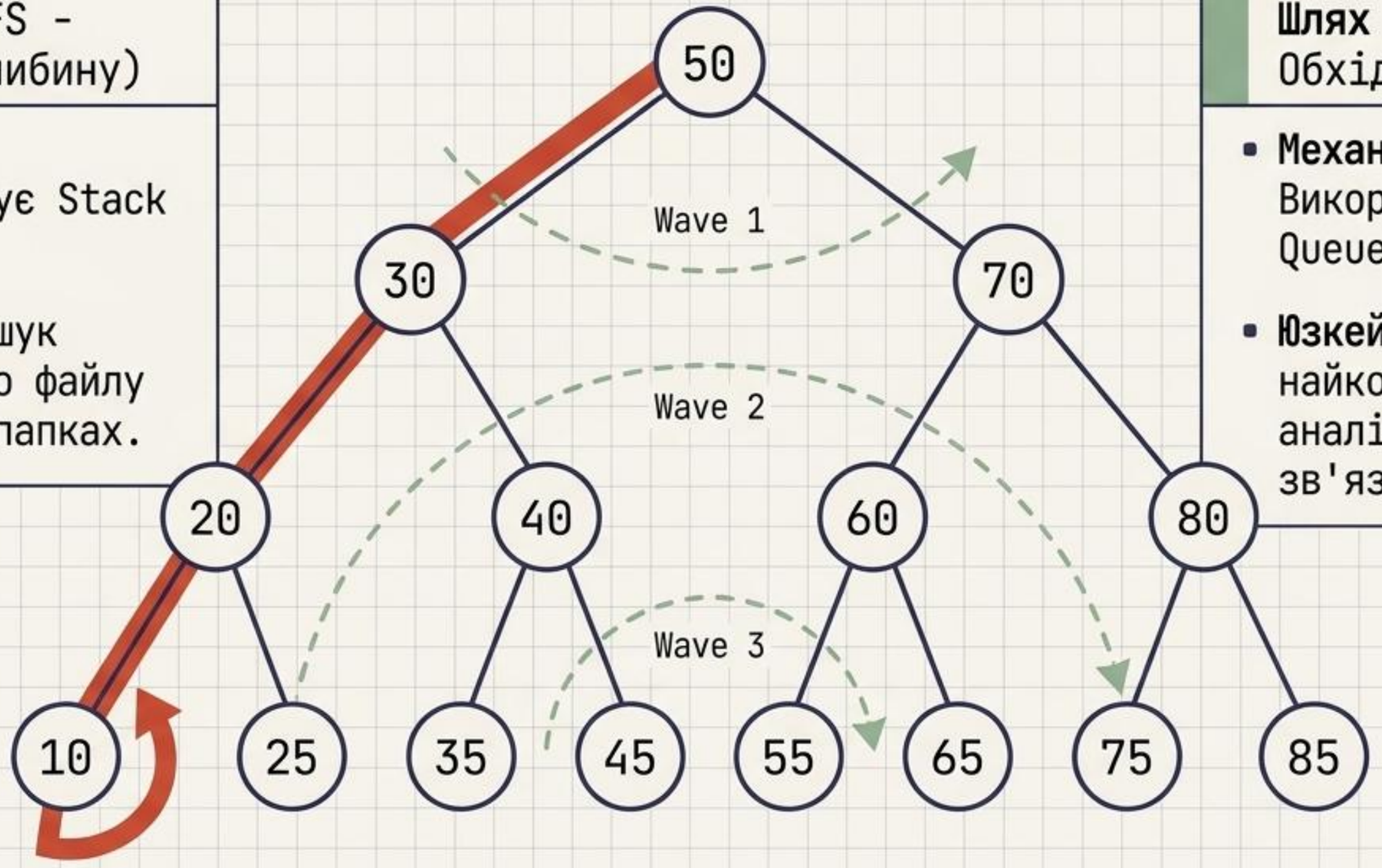
Складність: O(1) — одна миттєва операція. Дані не рухаються.

Gemini Notebook

Ієрархії: Дерева та алгоритми їх обходу

Шлях 1 (DFS - Обхід у глибину)

- **Механізм:** Використовує Stack (Стек).
- **Юзкейс:** Пошук конкретного файлу глибоко в папках.



Шлях 2 (BFS - Обхід у ширину)

- **Механізм:** Використовує Queue (Чергу).
- **Юзкейс:** Пошук найкоротшого шляху, аналіз найближчих зв'язків.

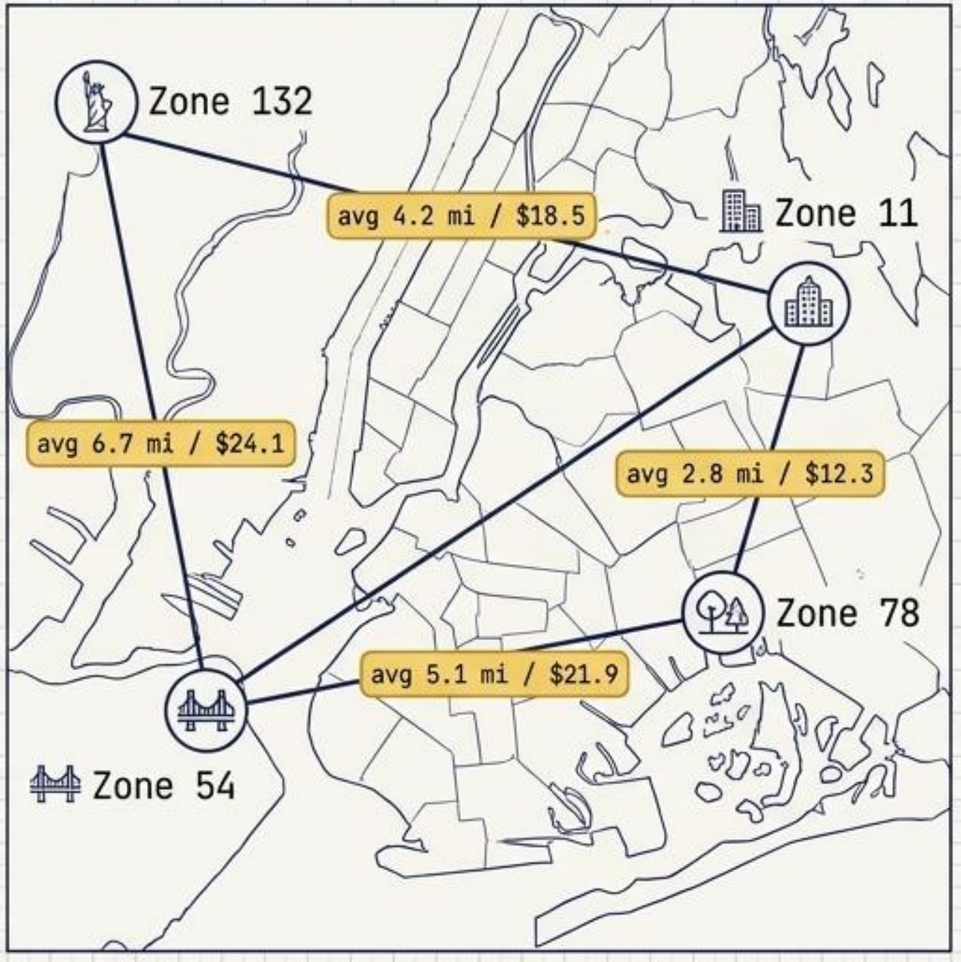
Gemini Notebook

Графи: Мережі реального світу

Що таке граф?

Граф зберігає реальність такою, якою вона є (перехрестя, зв'язки), не спотворюючи її у таблиці.

Вузол = Сутність,
Ребро = Зв'язок.



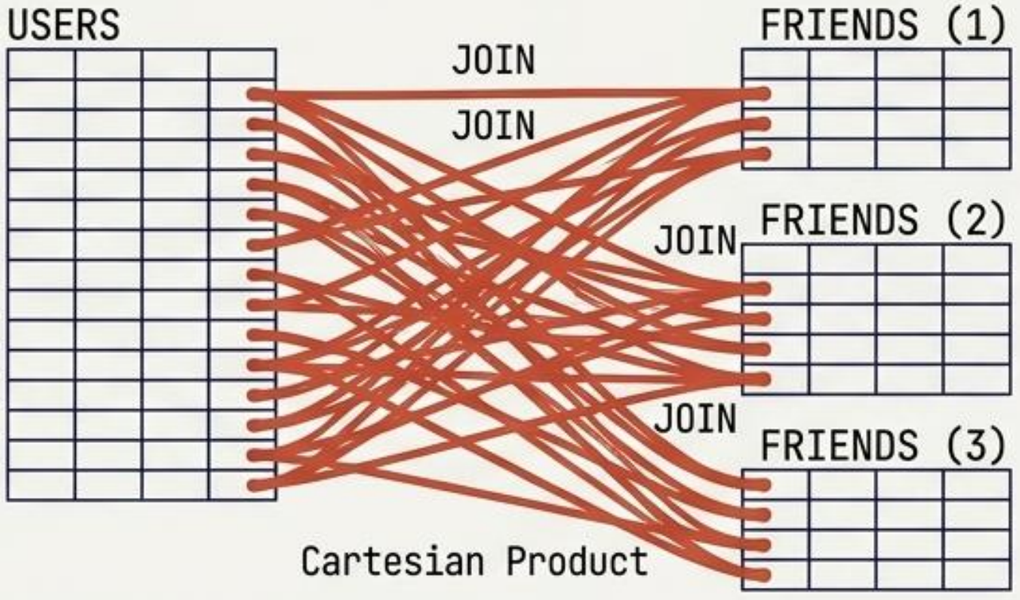
Алгоритм Дейкстри

На відміну від BFS (який рахує мінімум кроків), Дейкстра аналізує *вагу* ребер, щоб знайти найдешевший або найшвидший маршрут, навіть якщо він складається з більшої кількості переходів.

Gemini Notebook

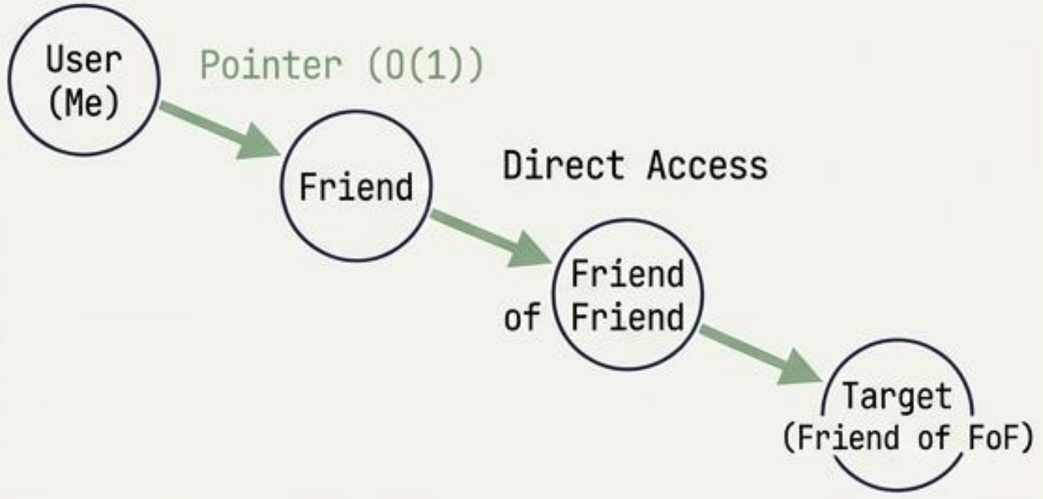
Проблема глибини: SQL проти Graph Traversal

SQL - Проблема



- **Механізм:** 4 важкі JOIN операції.
- **Результат:** Продуктивність деградує ЕКСПОНЕНЦІАЛЬНО. Час: 1500 секунд (на 1М користувачів).

Neo4j Graph - Рішення

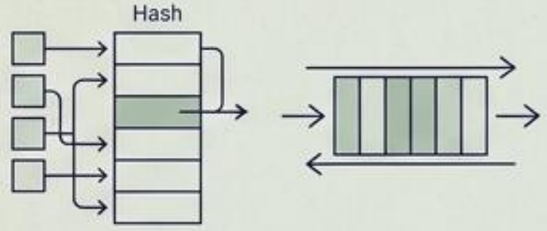


- **Механізм:** Index-Free Adjacency. Обхід ребра — це просто перехід за вказівником в пам'яті (O(1)).
- **Результат:** Той самий запит займає 1.3 секунди. (Ідеально для Fraud Detection).

Gemini Notebook

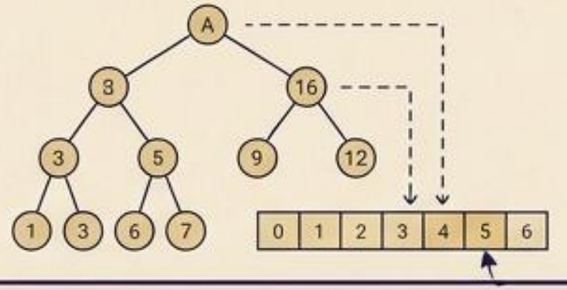
Матриця ухвалення рішень (Big-O Dashboard)

Телепортація / O(1)



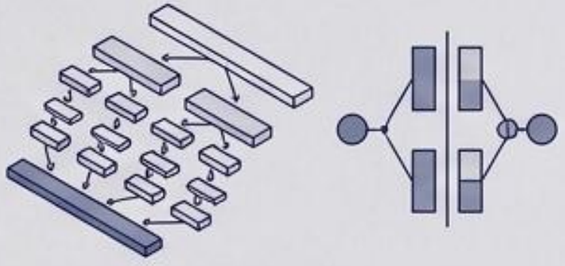
Хеш-таблиці (`dict`, `set`), Deque (на кінцях). Ідеально для точного пошуку.

Розділяй і володарюй / O(log n)



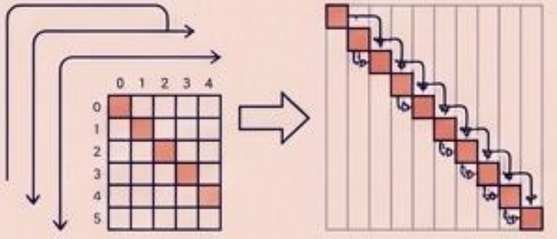
Бінарний пошук, Збалансовані дерева. Ідеально для баз даних.

Масштабовані операції / O(n log n)



QuickSort, MergeSort, Timsort. Стандарт для впорядкування хаосу.

Зона ризику / O(n^2)



Вкладені цикли, Bubble Sort, `list.pop(0)`. Уникати в production.

Алгоритми — це не синтаксис коду. Це фундаментальний спосіб мислення, який відрізняє працюючий скрипт від масштабованої архітектури.

Gemini Notebook